1. 项目概述

这是一个基于问卷的游戏推荐 Web 应用。

系统通过 **10～15 道选择题** 收集用户偏好，将用户答案映射为标签偏好权重，再从数据库中的游戏集合中召回候选游戏，按规则进行排序，最终返回 **3 个推荐结果**，并附上推荐理由与 Steam 商店链接。

第一阶段不接入 LLM，推荐系统必须依靠**规则系统本身独立成立**。

第二阶段再引入 LLM，用于对候选结果进行**结构化精排**和**推荐理由增强**。

第三阶段补测试与 CI。

第四阶段再完成部署。这个项目的核心目标不是“做一个自由聊天的 AI 产品”，而是做一个：

- **结构清楚**
- **逻辑可解释**
- **可扩展**
- **能逐步工程化**

的 AI 应用项目。

---

2. 领域词汇

Questionnaire

问卷题目。用户需要完成的 10～15 道选择题。

User Tag Profile

用户标签画像。系统根据用户的选项计算出的标签偏好权重集合。

Candidate Retrieval

候选召回。系统从游戏库中筛选出可能匹配用户偏好的游戏。

Rule Ranking

规则排序。基于标签权重和过滤逻辑对候选游戏进行打分排序。

Rule Reason

规则推荐理由。第一阶段通过模板生成的推荐说明。

LLM Rerank

LLM 精排。第二阶段中，LLM 只在候选集上进行排序增强，不参与全库推荐。

Debug Payload

调试信息。后端保留但前端默认不展示的解释性数据。

---

3. 项目目标

第一目标

先完成一个**不依赖 LLM 也能正常工作的游戏推荐系统。**

第二目标

在第一阶段架构不推翻的前提下，平滑接入 LLM。

第三目标

让项目具备较好的简历价值，不只是一个“会跑”的 demo，而是一个：

- 有合理数据库设计
- 有清晰 API 契约
- 有前后端分工
- 有可解释推荐逻辑
- 有 i18n
- 有后续测试 / CI / 部署扩展能力

的完整项目。

---

4. 四个阶段的开发规划

阶段 1：MVP（无 LLM 的可运行推荐系统）

阶段目标

完成一个最小但结构正确的推荐系统骨架：

- 前端问卷页
- 前端结果页
- 后端题目接口
- 后端推荐接口
- PostgreSQL 数据库
- 规则推荐逻辑
- 模板化推荐理由
- 中英文切换
- 后端 debug 信息保留

这一阶段完成后，系统即使**没有任何 AI 能力**，也应该已经可以正常使用。

阶段 2：接入 LLM 的结构化输出

阶段目标

在第一阶段的规则推荐基础上，引入 LLM 做：

- 候选集内精排
- 推荐理由增强

但必须遵守一个原则：

LLM 不是推荐系统本体，只是规则系统之后的增强层。

也就是说：

- 候选集由规则系统召回
- LLM 只能从候选集里选
- LLM 必须输出结构化 JSON
- 一旦输出不合法，立即 fallback 到规则推荐结果

阶段 3：补测试与 CI

阶段目标

把前两个阶段做成一个更像“工程项目”的系统：

- 后端单元测试
- API 集成测试
- 前端 E2E 测试
- GitHub Actions 自动化执行测试

这一阶段不要求你一开始就写完，但第一阶段开始时就要为它留结构。

阶段 4：部署

阶段目标

完成项目上线与可运行化：

- 前后端部署
- 数据库配置化
- 环境变量管理
- 健康检查
- 基础运行说明
- README 完整说明

---

5. 阶段 1 详细设计

5.1 功能范围

本阶段包含的功能

- 展示首页
- 展示问卷页
- 获取题目与选项
- 用户逐题作答
- 显示答题进度条
- 提交答案

- 提交时显示 loading
- 请求失败时显示错误提示
- 展示推荐结果
- 展示 3 个游戏推荐
- 展示游戏中英文名称
- 展示 Steam 链接
- 展示规则推荐理由
- 支持中英文切换
- 后端保留 debug 信息

本阶段不包含的功能

- 登录 / 注册
- 用户系统
- 收藏 / 历史记录
- 管理员后台
- 游戏管理页面
- 聊天式 AI
- 向量数据库
- Redis
- 推荐历史持久化
- 用户画像保存

5.2 页面设计

1）首页 /

用于展示项目介绍和开始按钮。

建议内容：

- 项目标题
- 一段简短介绍
- “开始测试”按钮

- 语言切换按钮
- 可选的“推荐方法说明”入口

2）问卷页 /quiz

用于完成题目填写。

页面应包含：

- 当前题目标题
- 当前题目的选项列表
- 当前进度条
- 上一题 / 下一题按钮
- 最后一题的提交按钮
- 提交时 loading 状态
- 错误提示区域
- 语言切换按钮

3）结果页 /result

用于展示推荐结果。

页面应包含：

- 推荐结果标题
- 3 张游戏推荐卡片
- 每张卡片内容： 中文名
  - 英文名
  - Steam 商店链接
  - 推荐理由
- “重新测试”按钮
- 语言切换按钮

4）可选说明页 /methodology

不是必须，但如果你想让项目显得更完整，可以加一个说明页，介绍：

- 推荐逻辑概述
- 标签权重是如何计算的
- 第一阶段不使用 LLM
- 第二阶段会如何引入 LLM

5.3 前端设计

技术栈

- React
- JavaScript
- React Router
- CSS / Tailwind（二选一，按你习惯）
- react-i18next（推荐）

前端核心状态

建议至少有这些状态：

- questions
- answers
- currentQuestionIndex
- isSubmitting
- submitError
- result

其中 answers 建议使用下面这种结构：

```
{
  "1": 11,
```

```
  "2": 24,
  "3": 35
}
```

也就是：

- key = question_id
- value = option_id

这样结构简单，便于提交给后端。

前端组件建议

首页

- HomePage

问卷页

- QuizPage
- ProgressBar
- QuestionCard
- OptionList
- ErrorMessage
- LanguageSwitcher

结果页

- ResultPage
- RecommendationList
- RecommendationCard
- LanguageSwitcher

前端交互要求

进度条

进度条显示当前已完成题目数 / 总题目数。

例如：

- 第 3 题 / 共 12 题
- 或百分比进度条

---

loading

点击提交后：

- 提交按钮 disabled
- 显示 loading 文案
- 防止重复提交

错误提示

接口失败时必须给出明确提示，例如：

- 获取题目失败
- 提交失败
- 系统繁忙，请稍后重试

注意：

前端只展示用户友好提示，真正的错误码由后端统一定义。

5.4 i18n 设计

你要求完整中英文 i18n，这里建议从一开始就把职责拆清楚。

前端负责翻译的内容

前端自己管理 UI 壳子的翻译，包括：

- 按钮文本

- 页面标题
- 错误提示
- loading 文案
- 导航文案
- 通用提示文字

例如：

- 开始测试 / Start Quiz
- 上一题 / Previous
- 下一题 / Next
- 提交 / Submit
- 重新测试 / Retry

后端负责提供的双语业务内容

后端返回以下业务数据的中英文：

- 题目标题
- 题目描述
- 选项文本
- 游戏名
- 标签名
- 推荐理由

这样做的原因是：

题目和推荐理由属于“业务内容”，不应该写死在前端 locale 文件里。

否则以后你改题目、改标签、改理由模板会很麻烦。

语言参数建议

前后端统一使用：

- zh
- en

后端接口支持通过 lang 参数控制返回语言。

例如：

- GET /questions?lang=zh
- POST /recommend 请求体中带 lang

5.5 后端设计

技术栈

- FastAPI
- SQLModel
- PostgreSQL
- Pydantic
- Alembic（用于数据库 migration）

技术选型说明

本项目后端 ORM 统一使用 **SQLModel**，不再采用 “SQLAlchemy / SQLModel 二选一” 的方案。

这样做的原因是：

- 对当前阶段更友好，学习成本比直接使用 SQLAlchemy ORM 更低
- 与 FastAPI、Pydantic 的使用风格更统一
- 足以满足本项目第一阶段和第二阶段的建模、查询与关系管理需求
- 底层仍基于 SQLAlchemy，后续如果需要更复杂能力，仍可逐步扩展

本项目数据库 migration 统一使用 **Alembic**。
也就是说：

- **数据模型定义：SQLModel**

- **数据库表结构迁移：Alembic**

第一阶段的核心接口

1）获取题目接口

GET /questions?lang=zh

作用：

- 返回题目列表
- 返回选项列表
- 按题目顺序排列
- 按当前语言返回业务文本

2）推荐接口

POST /recommend

作用：

- 接收用户答案
- 校验答案是否合法
- 计算用户标签画像
- 执行候选召回与排序
- 生成推荐理由
- 返回 top 3 推荐结果
- 返回 debug 信息

5.6 API 设计

GET /questions

请求示例

GET /questions?lang=zh

响应示例


```
{
  "questions": [
    {
      "id": 1,
      "code": "preferred_experience",
      "title": "你更喜欢哪种游戏体验？",
      "description": null,
      "sort_order": 1,
      "options": [
        {
          "id": 11,
          "code": "story_immersion",
          "text": "沉浸剧情",
          "sort_order": 1
        },
        {
          "id": 12,
          "code": "free_exploration",
          "text": "自由探索",
          "sort_order": 2
        }
      ]
    }
  ]
}
```


设计说明

- code 是内部稳定标识
- 前端只负责展示，不参与推荐逻辑

- 题目和选项必须按 sort_order 返回

POST /recommend

请求示例


```
{
  "answers": [
    { "question_id": 1, "option_id": 11 },
    { "question_id": 2, "option_id": 24 }
  ],
  "lang": "zh"
}
```


响应示例


```
{
  "recommendations": [
    {
      "game_id": 101,
      "name_zh": "巫师 3：狂猎",
      "name_en": "The Witcher 3: Wild Hunt",
      "steam_url": "https://store.steampowered.com/...",
      "score": 18.5,
      "reason": "你偏好剧情丰富、开放探索和角色成长，这款游戏在这些方面匹配度较高。",
      "matched_tags": [
        { "tag_code": "story_rich", "display_name": "剧情丰富", "weight": 5 },
        { "tag_code": "open_world", "display_name": "开放世界", "weight": 4 }
      ]
    }
  ],
  "debug": {
```

"user_tag_profile": [
{ "tag_code": "story_rich", "weight": 5 },
{ "tag_code": "open_world", "weight": 4 },
{ "tag_code": "horror", "weight": -3 }
],
"applied_filters": [
{ "type": "hard_exclude", "tag_code": "horror", "threshold": -2 }
],
"candidate_count_before_filter": 120,
"candidate_count_after_filter": 73
}
}

错误响应格式

建议统一为：

{
"error": {
"code": "INVALID_ANSWER_OPTION",
"message": "所选答案与题目不匹配。"
}
}

规范：

- 前端依赖 code
- message 用于用户展示
- message 可根据 lang 返回中英文

5.7 数据库设计

你已经确定：

- 不做用户表

- 不做管理员表
- 保留第六张表 option_tag_weights

这是合理的。

数据表总览（第一阶段核心 6 张表）

- games
- tags
- game_tags
- questions
- options
- option_tag_weights

为了给第二阶段留扩展位，建议从设计上知道未来还会有：

- recommendation_runs
- llm_audit_logs

但这两张表可以不是第一阶段必须实现的核心表。

1）games

```
CREATE TABLE games (
    id BIGSERIAL PRIMARY KEY,
    steam_app_id BIGINT UNIQUE,
    slug VARCHAR(255) UNIQUE,
    name_zh VARCHAR(255),
    name_en VARCHAR(255) NOT NULL,
    steam_url TEXT NOT NULL,
    short_description_zh TEXT,
    short_description_en TEXT,
    cover_image_url TEXT,
    release_date DATE,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
```

```
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

字段说明

- steam_app_id Steam 的应用 ID，方便未来做数据同步或去重。 slug
- 用于内部唯一标识和潜在 SEO / 前端路由扩展，第一阶段不是必须用到，但建议保留。
- name_zh, name_en 中英文游戏名分开存。不要把这类核心业务文本放进 JSON。
- steam_url 第一阶段前端展示推荐结果时必须要用。
- short_description_zh, short_description_en 第一阶段可以不用展示，但很适合后续扩展。
- is_active 用于停用某些不再参与推荐的游戏。说明：
- name_zh 可为空，因为有些游戏可能没有稳定中文名
- steam_url 第一阶段是必须字段
- is_active 用于未来停用无效游戏数据

2）tags

```
CREATE TABLE tags (
    id BIGSERIAL PRIMARY KEY,
    code VARCHAR(100) NOT NULL UNIQUE,
    name_zh VARCHAR(100) NOT NULL,
    name_en VARCHAR(100) NOT NULL,
    description_zh TEXT,
    description_en TEXT,
    category VARCHAR(100),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

字段说明

- code
  - 系统内部稳定标识，例如： story_rich
  - open_world
  - horror
  - cozy

后续推荐逻辑、测试、问卷映射都依赖这个字段，而不是中文名。

- category
  - 可用于将标签做简单分组，例如： theme
  - mechanic
  - pace
  - mood

第一阶段不是必须使用，但建议保留。

- code 是系统内部稳定标识
- 推荐逻辑和测试都应该依赖 code

3）game_tags

```
CREATE TABLE game_tags (
    id BIGSERIAL PRIMARY KEY,
    game_id BIGINT NOT NULL REFERENCES games(id) ON DELETE CASCADE,
    tag_id BIGINT NOT NULL REFERENCES tags(id) ON DELETE CASCADE,
    weight SMALLINT NOT NULL,
    source VARCHAR(100),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_game_tags UNIQUE (game_id, tag_id),
    CONSTRAINT chk_game_tags_weight CHECK (weight BETWEEN 1 AND 3)
);
```

字段说明

- weight

表示“该游戏和某标签的相关强度”。

  - 第一阶段建议只用 1 ~ 3： 1 = 弱相关
  - 2 = 中等相关
  - 3 = 强相关

- source
  - 可记录这条标签数据的来源，例如： rawg_import
  - manual_review
  - steamspy_mapping

第一阶段推荐逻辑不依赖它，但以后做数据清洗时会有帮助。

约束建议：

- unique(game_id, tag_id)

说明：

- 这里的 weight 表示“这个游戏和某标签的相关强度”
- 第一阶段建议用简单离散值，不要太细

4）questions

```
CREATE TABLE questions (
    id BIGSERIAL PRIMARY KEY,
    code VARCHAR(100) NOT NULL UNIQUE,
    title_zh VARCHAR(255) NOT NULL,
    title_en VARCHAR(255) NOT NULL,
    description_zh TEXT,
    description_en TEXT,
    sort_order INTEGER NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
```

);

字段说明

- code
  - 题目的内部稳定标识，例如： preferred_experience
  - difficulty_preference
  - multiplayer_preference
- sort_order
- 控制题目在问卷中的顺序。 is_active

方便以后停用某个题目而不是直接删除。

5）options

```
CREATE TABLE options (
    id BIGSERIAL PRIMARY KEY,
    question_id BIGINT NOT NULL REFERENCES questions(id) ON DELETE CASCADE,
    code VARCHAR(100) NOT NULL UNIQUE,
    text_zh VARCHAR(255) NOT NULL,
    text_en VARCHAR(255) NOT NULL,
    sort_order INTEGER NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_options_question_sort UNIQUE (question_id, sort_order)
);
```

字段说明

- question_id
- 指明该选项属于哪一道题。 code
  - 选项的内部稳定标识，例如： story_immersion
  - free_exploration
  - avoid_horror

- sort_order

控制选项展示顺序。

约束建议：

- unique(question_id, sort_order)

6）option_tag_weights

```
CREATE TABLE option_tag_weights (
    id BIGSERIAL PRIMARY KEY,
    option_id BIGINT NOT NULL REFERENCES options(id) ON DELETE CASCADE,
    tag_id BIGINT NOT NULL REFERENCES tags(id) ON DELETE CASCADE,
    weight SMALLINT NOT NULL,
    weight_type VARCHAR(50) NOT NULL DEFAULT 'soft',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_option_tag_weights UNIQUE (option_id, tag_id),
    CONSTRAINT chk_option_tag_weights_weight CHECK (weight BETWEEN -5 AND 5),
    CONSTRAINT chk_option_tag_weights_type CHECK (weight_type IN ('soft', 'hard_exclude'))
);
```

字段说明

- weight

表示用户选了某个选项之后，对某标签的偏好影响。 例如：

- story_rich +2
- open_world +1
- horror -3
- weight_type
    - 第一阶段建议保留两个值： soft：参与普通打分

    - hard_exclude：达到条件时参与排除逻辑

这样后续扩展不需要改表。

约束建议：

- unique(option_id, tag_id)
- weight 范围建议为 -5 ~ 5

说明：

- 这是用户侧标签偏好的来源
- weight_type 用于后续区分： soft
    - hard_exclude

第一阶段就建议把这个字段设计好，即使第一版只少量使用硬过滤

5.8 关系设计

games ↔ tags

多对多，通过 game_tags

questions → options

一对多

options ↔ tags

多对多，通过 option_tag_weights

5.9 第一阶段推荐逻辑

这是第一阶段最核心的部分。

Step 1：校验答案合法性

需要检查：

- 题目是否存在
- 选项是否存在
- 选项是否属于对应题目
- 是否有重复答案

- 是否达到要求的最少答题数

Step 2：生成用户标签画像

从 option_tag_weights 中读取映射并累加，形成：

```
{
  "story_rich": 4,
  "open_world": 3,
  "horror": -3
}
```

Step 3：执行硬过滤

根据强负向标签执行候选排除。

例如：

- 用户对 horror 明显负向
- 排除恐怖标签很强的游戏

第一阶段只需要少量硬过滤，不要做复杂规则。

Step 4：候选打分

建议公式：

总分 = Σ（用户标签偏好权重 × 游戏标签权重）

这已经足够作为第一阶段推荐逻辑。

不要一开始追求复杂数学模型。

Step 5：返回 top 3

按分数排序后返回前 3 个。

Step 6：生成规则推荐理由

第一阶段推荐理由用模板生成。

例如：

- 你偏好剧情丰富、开放探索和角色成长，因此这款游戏更符合你的选择倾向。
- 你倾向轻松休闲和低压力体验，这款游戏在这些方面匹配较好。

推荐理由依赖：

- 匹配度最高的几个标签
- 当前语言
- 模板规则

5.10 后端结构建议

建议目录结构如下：

```
backend/
  app/
    main.py
    api/
      routes/
        questions.py
        recommend.py
    schemas/
      common.py
      questions.py
      recommend.py
    models/
      game.py
      tag.py
      game_tag.py
      question.py
      option.py
```

```
        option_tag_weight.py
    services/
        question_service.py
        answer_validation_service.py
        profile_builder_service.py
        candidate_service.py
        ranking_service.py
        reason_service.py
        recommendation_service.py
    db/
        session.py
    core/
        config.py

  alembic/
  alembic.ini
```

结构说明

- `api/routes/`：定义 FastAPI 路由
- `schemas/`：定义 Pydantic 请求 / 响应模型
- `models/`：定义 SQLModel 数据模型
- `services/`：承载业务逻辑
- `db/session.py`：数据库连接与 Session 管理
- `alembic/`：数据库 migration 文件

设计原则

- route 层只处理请求与响应
- service 层处理业务逻辑
- schema 层定义输入输出
- model 层使用 SQLModel 定义数据库结构

- 数据库结构变更通过 Alembic 管理

这样做的原因是：

- 第二阶段接 LLM 更容易
- 第三阶段测试更容易
- 第四阶段部署更容易
- 对当前阶段的学习曲线也更友好

---

6. 阶段 2 详细设计：接入 LLM 结构化输出

6.1 阶段目标

在第一阶段的规则推荐已经成立的前提下，增加：

- LLM 候选精排
- LLM 推荐理由增强

但必须坚持：

LLM 只能增强，不可以取代规则系统。

6.2 第二阶段数据流

问卷

→ 用户标签画像

→ 规则候选召回

→ 规则排序

→ 取前 10 个候选

→ LLM 从前 10 中选出 3 个

→ 校验 LLM 输出是否合法

→ 若不合法则 fallback 到规则 top 3

→ 返回最终结果

6.3 LLM 的职责边界

LLM 允许做的事：

- 从候选集中挑选更合适的 3 个
- 生成更自然的推荐理由
- 在不改变事实的前提下增强表达

LLM 不允许做的事：

- 从候选集之外推荐新游戏
- 自由联想不存在的游戏
- 取代规则召回
- 输出非结构化自由文本作为最终结果

6.4 LLM 输入建议

输入给 LLM 的内容只包括：

- 用户标签画像摘要
- 规则排序后的 top 10 候选
- 每个候选的核心标签
- 当前语言

不要把整个游戏数据库发给 LLM。

6.5 LLM 输出格式

建议为固定 JSON：

```
{
  "selected_game_ids": [101, 205, 88],
  "reasons": [
    {
      "game_id": 101,
      "reason": "..."
    },
```

```
    {
      "game_id": 205,
      "reason": "..."
    },
    {
      "game_id": 88,
      "reason": "..."
    }
  ]
}
```

6.6 第二阶段的 fallback 规则

如果 LLM 出现以下情况，直接 fallback：

- 不是合法 JSON
- 数量不对
- 出现重复 game_id
- 推荐了候选集之外的游戏
- reason 缺失
- 响应超时
- API 调用失败

fallback 后：

- 仍返回规则 top 3
- 推荐理由可继续用模板理由

6.7 第一阶段就要预留的扩展位

第一阶段就建议你把以下两个服务抽象出来：

rerank_service

- 第一阶段：直接取规则 top 3
- 第二阶段：规则 top 10 + LLM 精排

reason_service

- 第一阶段：模板理由
- 第二阶段：LLM 理由优先，失败则模板理由 fallback

这样第二阶段几乎不需要推翻第一阶段结构。

---

7. 阶段 3：测试与 CI（方向性设计）

这一阶段先不展开太细，但现在就要为它留结构。

7.1 后续要补的测试类型

单元测试

主要测：

- 答案校验
- 用户标签画像生成
- 硬过滤逻辑
- 打分逻辑
- 模板理由生成

集成测试

主要测：

- GET /questions
- POST /recommend
- 数据库查询与响应格式
- 错误处理

E2E 测试

主要测：

- 打开问卷页
- 完成作答

- 看到进度条变化
- 提交时出现 loading
- 返回结果
- 出错时出现提示

7.2 为什么现在就要提前考虑测试

因为如果你现在把推荐逻辑都写进路由函数里，后面测试会很难写。

所以第一阶段就必须：

- route 薄
- service 清晰
- schema 固定
- 数据模型稳定

7.3 CI 方向

后续可以用 GitHub Actions 做：

- 后端测试
- 前端构建检查
- E2E 测试
- 合并前自动检查

这一阶段现在只需要你保证：

- 启动命令清晰
- 测试命令清晰
- 环境变量清晰
- 数据库迁移命令清晰

---

8. 阶段 4：部署（方向性设计）

8.1 后续部署目标

- 前端可访问
- 后端 API 可访问
- PostgreSQL 可连接
- 环境变量可配置
- 项目有 README 运行说明

8.2 现在就应该提前考虑的点

第一阶段就应做到：

- 数据库连接字符串不写死
- 前端 API 地址可配置
- 语言和业务数据不依赖本地硬编码
- 后端具备基本健康检查思路
- 前后端目录清晰

---

9. 当前推荐的最终方案

基于你现在的时间和目标，我认为最合理的版本是：

阶段 1

做一个不依赖 LLM 的完整推荐系统：

- React + JavaScript

- FastAPI

- SQLModel

- PostgreSQL

- Alembic

- 2 个核心接口

- 6 张核心表

- 进度条

- loading
- 错误提示
- 完整中英文 i18n
- 规则推荐理由

- debug 后端保留阶段 2

在不推翻第一阶段结构的前提下接入 LLM：

- 候选集内精排
- 结构化 JSON 输出
- 严格校验
- fallback 到规则系统

阶段 3

补工程化：

- 单元测试
- API 测试
- E2E 测试
- GitHub Actions

阶段 4

部署上线：

- 环境变量
- 前后端部署
- 数据库配置
- README 运行说明